C++ Code Walkthrough

Linear Algebra Analyzer — Full Source Explanation

`linalg.h` · `linalg.cpp` · `main.cpp`

984 lines total · C++17 · Standard Library only

Sarvesh S

IIT Madras — `ed23b042@smail.iitm.ac.in`

This document explains *exactly how the code works* — every function, every loop, every design choice. Theory is covered separately in `linear_algebra_doc.pdf`.

May 28, 2026

0 Contents

1 Project Architecture

1.1 File layout and responsibilities

File	Responsibility	Lines
`linalg.h`	Class declaration, result-type structs, all `inline` predicates	129
`linalg.cpp`	Every algorithm implementation	637
`main.cpp`	Interactive CLI, formatted output	218

1.2 The `Matrix` class

Everything lives in one class, `Matrix`. Its public data members are:

```
int m, n;                                   // rows, columns
vector<vector<double>> A;                   // A[i][j] = element at row i, col j
static constexpr double EPS = 1e-9;    // global zero threshold
```

`A` is a *row-major* 2-D vector: `A[i]` is the i-th row as a `vector<double>`. Swapping two rows (`std::swap(A[i], A[j])`) is therefore $O(1)$ — it just swaps two pointers, not all the data. This is exploited heavily in Gauss elimination and LUP.

1.3 Why result structs live *outside* the class

The three result types — `RREFResult`, `EigenResult`, `JordanInfo` — each contain a `Matrix` member:

```
struct RREFResult { Matrix R; vector<int> pivot_cols; int rank; };
```

In C++, a nested struct defined *inside* a class body is parsed before the class is complete, so the incomplete type `Matrix` cannot appear as a value member. The fix: forward-declare the structs before the class so their names are visible inside the class for return types, then define them *after* the class is complete:

```
struct RREFResult;    // forward declaration
class Matrix { ... RREFResult rref() const; ... };
struct RREFResult { Matrix R; ... };    // full definition, Matrix is complete
```

1.4 C++17 structured bindings

Throughout the code, return values are destructured with `auto`:

```
auto [R, pivot_cols, rank_] = A.rref();    // RREFResult
auto [L, U, P]              = A.LUP();     // tuple<Matrix,Matrix,Matrix>
auto [Q, Rm]                = A.QR();      // pair<Matrix,Matrix>
auto [U_s, S_s, V_s]        = A.SVD();     // tuple<Matrix,Matrix,Matrix>
```

`RREFResult` is a C++ aggregate (public members, no constructor), and C++17 aggregate structured bindings work on those exactly like `tuple`.

2 Constructors and Static Helpers

```
Matrix::Matrix(int m_, int n_, double val)
    : m(m_), n(n_), A(m_, vector<double>(n_, val)) {}
```

The initialiser list builds `A` as `m_` rows each containing `n_` copies of `val`. Calling `Matrix(3,4,0.0)` gives a 3×4 zero matrix. Default `val=0.0` so `Matrix(3,4)` also gives zeros.

```
Matrix::Matrix(const vector<vector<double>>& data)
    : m(data.size()), n(data.empty() ? 0 : data[0].size()), A(data) {}
```

Allows direct construction from a 2-D `vector`, e.g. `Matrix({{1,2},{3,4}})`.

```
Matrix Matrix::eye(int n) {
    Matrix I(n, n, 0.0);
    for (int i = 0; i < n; i++) I.A[i][i] = 1.0;
    return I;
}
```

`eye(n)` starts from a zero matrix, then sets the diagonal. `Matrix::zeros(r,c)` just calls the value constructor. `fromInput` reads from `std::cin` row-by-row.

3 Basic Operations

3.1 Arithmetic operators

```
Matrix Matrix::operator*(const Matrix& B) const {
    if (n != B.m) throw runtime_error("*: size mismatch");
    Matrix C(m, B.n, 0.0);
    for (int i = 0; i < m; i++)
        for (int k = 0; k < n; k++)
            for (int j = 0; j < B.n; j++)
                C.A[i][j] += A[i][k] * B.A[k][j];
    return C;
}
```

The loop order is `i-k-j` rather than the textbook `i-j-k`. The inner loop `j` walks along a row of `B` and a row of `C` with `A[i][k]` fixed. This is better for CPU cache: `B.A[k]` and `C.A[i]` are both contiguous in memory.

3.2 Transpose

```
Matrix Matrix::T() const {
    Matrix C(n, m);
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            C.A[j][i] = A[i][j];   // note the flip
    return C;
}
```

Returns a *new* matrix of size $n \times m$. Chaining is possible: `A.T() * A` computes $A^T A$.

3.3 print()

```cpp
void Matrix::print(const string& label, int prec) const {
    if (!label.empty()) cout << label << ":\n";
    for (int i = 0; i < m; i++) {
        cout << "  [";
        for (int j = 0; j < n; j++) {
            double v = (abs(A[i][j]) < EPS) ? 0.0 : A[i][j];
            cout << setw(prec+6) << fixed << setprecision(prec) << v;
        }
        cout << " ]\n";
    }
}
```

The line `double v = (abs(...) < EPS) ? 0.0 : A[i][j]` cleans up floating-point noise before printing — values smaller than 10^{-9} are shown as `0.0000` instead of `-0.0000` or `1.23e-14`.

4 RREF — The Central Engine

`rref()` is the most-called function in the project. It is used by `rank`, `nullSpace`, `columnSpace`, `rowSpace`, `inv`, and `jordanForm`.

```cpp
RREFResult Matrix::rref() const {
    Matrix R = *this;           // work on a copy; don't modify A
    vector<int> pivot_cols;
    int cur_row = 0;

    for (int col = 0; col < n && cur_row < m; col++) {
        // (1) Find pivot: largest |entry| in column, from cur_row down
        int piv = -1;
        double best = EPS;
        for (int r = cur_row; r < m; r++) {
            if (abs(R.A[r][col]) > best) { best = abs(R.A[r][col]); piv
                = r; }
        }
        if (piv == -1) continue;    // entire column is zero -> free
            variable

        // (2) Swap pivot row to cur_row
        swap(R.A[cur_row], R.A[piv]);

        // (3) Scale: make leading entry exactly 1
        double sc = R.A[cur_row][col];
        for (int j = 0; j < n; j++) R.A[cur_row][j] /= sc;

        // (4) Eliminate: zero out ALL other rows in this column
        for (int r = 0; r < m; r++) {
            if (r == cur_row) continue;
            double f = R.A[r][col];
            if (abs(f) < EPS) continue;
            for (int j = 0; j < n; j++) R.A[r][j] -= f * R.A[cur_row][j
                ];
        }
        pivot_cols.push_back(col);
        cur_row++;
    }
```

```
    // (5) Snap near-zero to exact zero
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            if (abs(R.A[i][j]) < EPS) R.A[i][j] = 0.0;

    return {R, pivot_cols, (int)pivot_cols.size()};
}
```

Suppose $A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 8 \\ 0 & 1 & 2 & 3 \end{pmatrix}$:

- **col=0:** Largest in col 0 is row 1 (`|2|>|1|>|0|`). Swap rows 0↔1. Scale: divide row 0 by 2. Eliminate: row 1 − = 1· row 0; row 2 unchanged. Record `pivot_cols = {0}`, `cur_row = 1`.
- **col=1:** Best below cur_row=1 is row 2 ($|1| > |0|$). Swap rows 1↔2. Scale by 1. Eliminate all others. Record `pivot_cols = {0,1}`, `cur_row = 2`.
- **col=2,3:** All entries at/below cur_row=2 are zero. `piv = -1` ⇒ `continue`.

Result: rank 2, free columns $\{2, 3\}$.

Why Gauss-Jordan (not Gaussian)? Plain Gaussian elimination gives upper triangular U. Gauss-Jordan also eliminates *above* the pivot (step 4 iterates over *all* rows), giving reduced row echelon form where each pivot column has exactly one non-zero entry (the leading 1). This makes reading off the null-space vectors trivial.

5 LUP Decomposition

```
tuple<Matrix,Matrix,Matrix> Matrix::LUP() const {
    int nn = m;
    Matrix L = eye(nn), U = *this, P = eye(nn);

    for (int k = 0; k < nn; k++) {
        // (1) Partial pivot: find row with largest |entry| in column k
        int piv = k; double best = abs(U.A[k][k]);
        for (int i = k+1; i < nn; i++)
            if (abs(U.A[i][k]) > best) { best = abs(U.A[i][k]); piv = i;
                }

        if (piv != k) {
            swap(U.A[k], U.A[piv]);     // swap in U
            swap(P.A[k], P.A[piv]);     // mirror swap in P
            // Swap ONLY the already-filled part of L (columns 0..k-1)
            for (int j = 0; j < k; j++) swap(L.A[k][j], L.A[piv][j]);
        }
        if (abs(U.A[k][k]) < EPS) continue;  // singular column

        // (2) Compute multipliers and eliminate below the diagonal
        for (int i = k+1; i < nn; i++) {
            L.A[i][k] = U.A[i][k] / U.A[k][k];   // store multiplier in
                L
```

```
            for (int j = k; j < nn; j++)
                U.A[i][j] -= L.A[i][k] * U.A[k][j];
        }
    }
    return {L, U, P};
}
```

How PA

- U starts as a copy of A. At step k, rows $k+1, \ldots, n$ below the diagonal in column k are zeroed out.
- L starts as I. The multiplier $\ell_{ik} = U_{ik}/U_{kk}$ that was used to zero U_{ik} is stored in L_{ik}.
- P starts as I. Every row swap is mirrored in P. After the loop, P encodes the row permutation so $PA = LU$ holds exactly.
- The partial swap of L (`for (int j = 0; j < k; j++)`) is necessary because the previously computed multipliers in L must also be permuted consistently.

LUP vs RREF: LUP is used only for square matrices to compute `det` and display L, U, P. Everything else (subspaces, rank, inverse) uses `rref` because Gauss-Jordan handles non-square matrices naturally.

6 Determinant

```
double Matrix::det() const {
    Matrix U = *this;     // work on a copy
    int swaps = 0;
    for (int k = 0; k < m; k++) {
        int piv = k; double best = abs(U.A[k][k]);
        for (int i = k+1; i < m; i++)
            if (abs(U.A[i][k]) > best) { best = abs(U.A[i][k]); piv = i;
                }
        if (piv != k) { swap(U.A[k], U.A[piv]); swaps++; }
        if (abs(U.A[k][k]) < EPS) return 0.0;    // singular -> det = 0

        for (int i = k+1; i < m; i++) {
            double f = U.A[i][k] / U.A[k][k];
            for (int j = k; j < m; j++) U.A[i][j] -= f * U.A[k][j];
        }
    }
    double d = (swaps % 2 == 0) ? 1.0 : -1.0;
    for (int i = 0; i < m; i++) d *= U.A[i][i];
    return d;
}
```

This is forward-only Gaussian elimination (not full RREF), just enough to get the upper-triangular form whose diagonal product gives $\det(A)$. **Why separate from `LUP`?** LUP also tracks L and P, costing extra memory. `det` needs only U and a swap count.

The sign rule: each row swap changes the sign of the determinant, so $\det(A) = (-1)^s \prod_i U_{ii}$ where s is the total number of swaps.

7 Matrix Inverse via Augmented Gauss-Jordan

```cpp
Matrix Matrix::inv() const {
    int nn = m;
    Matrix aug(nn, 2*nn, 0.0);     // build [A | I]
    for (int i = 0; i < nn; i++) {
        for (int j = 0; j < nn; j++) aug.A[i][j] = A[i][j];
        aug.A[i][nn+i] = 1.0;      // right half = identity
    }
    // Apply Gauss-Jordan to the full 2n-column augmented matrix
    int cur = 0;
    for (int col = 0; col < nn && cur < nn; col++) {
        int piv = -1; double best = EPS;
        for (int r = cur; r < nn; r++)
            if (abs(aug.A[r][col]) > best) { ... piv = r; }
        swap(aug.A[cur], aug.A[piv]);
        // scale and eliminate just as in rref(), but on all 2n columns
        ...
    }
    // Extract right half: that is now A^{-1}
    for (int i = 0; i < nn; i++)
        for (int j = 0; j < nn; j++)
            inv_mat.A[i][j] = aug.A[i][nn+j];
    return inv_mat;
}
```

The augmented matrix $[A \mid I]$ is passed through Gauss-Jordan. Every row operation applied to A is simultaneously applied to I. When A becomes I (left half), the right half has become A^{-1}. Every elimination step operates on *all* $2n$ *columns* of `aug`, which is why the inner loop uses `j < 2*nn`.

8 QR Decomposition (Modified Gram-Schmidt)

```cpp
pair<Matrix,Matrix> Matrix::QR() const {
    Matrix Q = zeros(m, n);    // Q has orthonormal columns
    Matrix R = zeros(n, n);    // R is upper triangular
    Matrix U = *this;          // working copy of A; columns get modified

    for (int j = 0; j < n; j++) {
        // (1) Norm of current j-th column of U
        double norm_sq = 0.0;
        for (int i = 0; i < m; i++) norm_sq += U.A[i][j] * U.A[i][j];
        double norm = sqrt(norm_sq);
        if (norm < EPS) continue;   // linearly dependent column: skip

        R.A[j][j] = norm;
        for (int i = 0; i < m; i++) Q.A[i][j] = U.A[i][j] / norm;

        // (2) KEY: project out Q[:,j] from ALL remaining columns of U
            immediately
        for (int k = j+1; k < n; k++) {
            double dot = 0.0;
            for (int i = 0; i < m; i++) dot += Q.A[i][j] * U.A[i][k];
            R.A[j][k] = dot;
            for (int i = 0; i < m; i++) U.A[i][k] -= dot * Q.A[i][j];
        }
```

```
    }
    return {Q, R};
}
```

Classical GS: For each column j, subtract projections onto $q_1, q_2, \ldots, q_{j-1}$ (the *original* unnormalized vectors from the previous round):

$$u_j = a_j - \sum_{i<j} \frac{a_j \cdot q_i}{\|q_i\|^2} q_i$$

This reads from `A[i][j]` (unchanged) and subtracts at the end.
Modified GS (what we use): After computing q_j, *immediately* modify all remaining columns of U by subtracting their projection onto q_j. When we later process column k, it has already had $q_1, \ldots, q_j$ projected out. In exact arithmetic, both give the same result. But floating-point errors accumulate differently: MGS reuses the *updated* $U[:, k]$ (which has less of q_j in it), so each step starts from a vector that is already more orthogonal. This is why MGS is preferred in practice.

How R is filled:

- $R_{jj} = \|u_j\|$ (the diagonal entry records the norm before normalization).
- $R_{jk} = q_j \cdot U[:, k]$ for $k > j$ (the inner product with the already-normalized q_j, stored before subtracting).
- $R_{jk} = 0$ for $k < j$ (never written; `zeros(n,n)` default).

9 Gram-Schmidt Orthonormalization

`gramSchmidt()` is different from `QR`: it operates on the columns of `*this` (which may already be a basis for a subspace) and returns only the linearly independent columns, orthonormalized. It is used by all four `orth*Space()` methods.

```
Matrix Matrix::gramSchmidt() const {
    vector<vector<double>> basis;   // growing list of orthonormal
       vectors
    int ncols = 0;
    Matrix Q(m, n, 0.0);

    for (int j = 0; j < n; j++) {
        vector<double> v(m);
        for (int i = 0; i < m; i++) v[i] = A[i][j];  // copy column j

        // Subtract projections onto all previously found orthonormal
           vectors
        for (auto& u : basis) {
            double dot = 0.0;
            for (int i = 0; i < m; i++) dot += u[i] * v[i];
            for (int i = 0; i < m; i++) v[i] -= dot * u[i];
        }
        double norm = 0.0;
        for (int i = 0; i < m; i++) norm += v[i]*v[i];
        norm = sqrt(norm);
```

```
        if (norm < EPS) continue;     // linearly dependent -> discard

        for (int i = 0; i < m; i++) v[i] /= norm;
        basis.push_back(v);
        for (int i = 0; i < m; i++) Q.A[i][ncols] = v[i];
        ncols++;
    }
    // Return a matrix with exactly ncols orthonormal columns
    Matrix result(m, ncols);
    for (int i = 0; i < m; i++)
        for (int j = 0; j < ncols; j++)
            result.A[i][j] = Q.A[i][j];
    return result;
}
```

gramSchmidt() QR()

QR always returns Q and R of the same size as the input and is designed for full-matrix factorization. gramSchmidt is simpler: it discards dependent columns and returns a compact result (fewer columns than the input if the subspace has lower dimension). Used as: `nullSpace().gramSchmidt()` → orthonormal null-space basis.

10 Eigenvalue Algorithms

10.1 _eigenSymmetric() — Unshifted QR Iteration

```
EigenResult Matrix::_eigenSymmetric() const {
    int nn = m;
    Matrix T_ = *this;    // will be driven to diagonal form
    Matrix V  = eye(nn);  // accumulates the product Q_1 Q_2 Q_3 ...

    for (int iter = 0; iter < 12000 * nn; iter++) {
        // Convergence check: off-diagonal Frobenius norm
        double off = 0.0;
        for (int i = 0; i < nn; i++)
            for (int j = 0; j < nn; j++)
                if (i != j) off += T_.A[i][j] * T_.A[i][j];
        if (sqrt(off) < EPS * nn) break;

        auto [Q, R] = T_.QR();     // factor T_ = QR
        T_ = R * Q;                // T_{k+1} = R_k Q_k = Q_k^T T_k Q_k
        V  = V * Q;                // V accumulates: V = Q_1 Q_2 ... Q_k
    }
    vector<double> vals(nn), ivals(nn, 0.0);
    for (int i = 0; i < nn; i++) vals[i] = T_.A[i][i];
    return {vals, ivals, V};
}
```

What the iteration does at each step

1. Compute $T_k = Q_k R_k$ (QR factorization).
2. Set $T_{k+1} = R_k Q_k$. Since $T_k = Q_k R_k \Rightarrow R_k = Q_k^T T_k$, we have $T_{k+1} = Q_k^T T_k Q_k$ — a similarity transformation. Eigenvalues are preserved.
3. Accumulate $V \leftarrow V Q_k$. After convergence V contains the eigenvectors as its columns.

Convergence: for a symmetric matrix, T_k converges to a diagonal matrix. The off-diagonal Frobenius norm $\|T_k - \mathrm{diag}(T_k)\|_F$ decreases monotonically. The loop breaks when it falls below εn. **Max iterations:** $12000n$ is generous; for a 3×3 matrix this is 36 000 iterations, but convergence typically happens in under 300.

Why no Rayleigh shift? A common speed-up is to subtract μI (where $\mu = T_{k,nn-1,nn-1}$) before the QR step and add it back after. When μ is exactly an eigenvalue (e.g., a matrix with repeated eigenvalue 2 at the bottom-right), $T_k - \mu I$ becomes *singular*: one column has near-zero norm in QR and the corresponding Q column is never computed, leaving a zero column in V. The unshifted version avoids this completely.

10.2 `_eigenGeneral()` — Faddeev-LeVerrier + Durand-Kerner

This function handles non-symmetric matrices, which may have complex eigenvalues. It has two phases.

10.2.1 Phase 1: Faddeev-LeVerrier — computing the characteristic polynomial

```
vector<double> coeff(nn + 1);
coeff[nn] = 1.0;          // leading coefficient is always 1
Matrix B = eye(nn);
for (int k = 1; k <= nn; k++) {
    B = (*this) * B;      // B <- A * B_{k-1}   (a fresh matrix multiply)
    double tr = 0.0;
    for (int i = 0; i < nn; i++) tr += B.A[i][i];   // trace of current B
    coeff[nn - k] = -tr / k;
    if (k < nn)
        for (int i = 0; i < nn; i++) B.A[i][i] += coeff[nn - k];  // B <- B + c*I
}
```

After the loop, `coeff[0..n]` hold the coefficients of $p(\lambda) = \lambda^n + c_{n-1}\lambda^{n-1} + \cdots + c_0$. The recurrence is: $c_{n-k} = -\operatorname{tr}(A \cdot B_{k-1})/k$ and $B_k = AB_{k-1} + c_{n-k}I$. This uses n matrix multiplications.

10.2.2 Phase 2: Durand-Kerner — finding all n roots simultaneously

```
// Lambda for polynomial evaluation using Horner's method
auto poly_eval = [&](C z) -> C {
    C r(coeff[nn]);                                  // start with leading 1
    for (int k = nn - 1; k >= 0; k--) r = r * z + C(coeff[k]);
```

```
    return r;                                                    // p(z)
};

// Initial positions: n points on a circle of radius r0 (Cauchy bound)
double cauchy = 0.0;
for (int k = 0; k < nn; k++)
    cauchy = max(cauchy, abs(coeff[k] / coeff[nn]));
double r0 = 1.0 + cauchy;
vector<C> roots(nn);
for (int k = 0; k < nn; k++) {
    double ang = 2.0 * M_PI * k / nn + 0.17;
    roots[k] = C(r0 * cos(ang), r0 * sin(ang));
}

for (int iter = 0; iter < 1000 * nn; iter++) {
    double max_chg = 0.0;
    vector<C> nr(nn);
    for (int i = 0; i < nn; i++) {
        // Denominator: product of (z_i - z_j) for all j != i
        C denom(1.0);
        for (int j = 0; j < nn; j++)
            if (j != i) denom *= (roots[i] - roots[j]);
        if (abs(denom) < 1e-15) { nr[i] = roots[i]; continue; }  //
            guard
        C corr = poly_eval(roots[i]) / denom;
        nr[i] = roots[i] - corr;          // Weierstrass correction
        max_chg = max(max_chg, abs(corr));
    }
    roots = nr;
    if (max_chg < 1e-12) break;
}
```

How Durand-Kerner works step by step

- **Polynomial evaluation:** `poly_eval` uses Horner's method $(\ldots((c_n z + c_{n-1})z + c_{n-2})\ldots)z + c_0)$ which is numerically stable and requires only n multiplications.
- **Initial guess radius `r0`:** the Cauchy bound $1 + \max_k |c_k/c_n|$ guarantees all roots lie *inside* the circle $|z| \le r_0$. Starting outside ensures no roots are missed.
- **Initial angle offset `0.17`:** placing roots at exact equally-spaced angles would cause symmetry that makes the denominator product zero; the `0.17` radian offset breaks this.
- **The correction** $p(z_i)/\prod_{j\neq i}(z_i - z_j)$ is the Newton step for the deflated polynomial $p(z)/\prod_{j\neq i}(z - z_j)$, whose unique root near z_i is the actual eigenvalue. All n corrections are computed *before* any update (stored in `nr`), making it a Jacobi-style simultaneous update.
- **Guard `if (abs(denom) < 1e-15)`:** if two initial guesses are identical (very rare), the denominator would be zero. We skip the update and let the iteration sort them out.

10.2.3 Phase 3: Classify and clean up eigenvalues

```
// Classify: imaginary part is "zero" if |Im| < 1e-4 * max(1, |Re|)
for (int i = 0; i < nn; i++) {
    double re = roots[i].real(), im = roots[i].imag();
```

```
    bool real_eig = abs(im) < max(1e-6, 1e-4 * abs(re));
    real_vals[i] = (abs(re) < 1e-9) ? 0.0 : re;
    imag_vals[i] = real_eig ? 0.0 : im;
}
```

For repeated real eigenvalues (e.g., $\lambda = 2$ with multiplicity 3), Durand-Kerner may not fully converge and leaves tiny imaginary parts $\sim 10^{-5}$. The *relative* threshold `1e-4 * |re|` catches these (for $\lambda \approx 2$, the threshold is 2×10^{-4}, larger than 10^{-5}) without misclassifying genuinely complex eigenvalues (where $|\text{Im}| \sim |\text{Re}|$).

10.2.4 Phase 4: Eigenvectors via `nullSpace`

```
Matrix V_ = eye(nn);  // fallback if all eigenvalues are complex
for (int k = 0; k < nn; k++) {
    if (abs(imag_vals[k]) > 1e-7) continue;  // skip complex eigenvalues
    double lam = real_vals[k];
    Matrix A_lam = *this;
    for (int i = 0; i < nn; i++) A_lam.A[i][i] -= lam;  // A - lambda*I
    Matrix ns = A_lam.nullSpace();   // RREF -> free variable vectors
    if (ns.n > 0)
        for (int i = 0; i < nn; i++) V_.A[i][k] = ns.A[i][0];
}
```

For each real eigenvalue λ, form $(A - \lambda I)$ by subtracting λ from the diagonal, then call `nullSpace()` which runs RREF to find the null-space basis. The first basis vector is stored as the k-th column of V_{-}.

10.3 `eigen()` — the dispatcher

```
EigenResult Matrix::eigen() const {
    if (!isSquare()) throw runtime_error("eigen: not square");
    if (isSymmetric()) return _eigenSymmetric();
    return _eigenGeneral();
}
```

`isSymmetric()` checks $|A_{ij} - A_{ji}| < \varepsilon$ for all entries. For symmetric matrices the unshifted QR iteration is used (guaranteed real eigenvalues, correct eigenvectors). For non-symmetric matrices, Faddeev-LeVerrier + Durand-Kerner handles the general case including complex eigenvalues.

11 SVD

```
tuple<Matrix,Matrix,Matrix> Matrix::SVD() const {
    Matrix AtA = T() * (*this);         // A^T A  (n x n, symmetric PSD)
    auto er = AtA._eigenSymmetric();    // eigenvalues in er.real_vals,
        eigenvectors in er.vecs
    int rr = min(m, n);

    // Sort eigenpairs by eigenvalue descending (largest sigma first)
    vector<pair<double,int>> sp;
    for (int i = 0; i < n; i++) sp.push_back({er.real_vals[i], i});
    sort(sp.begin(), sp.end(), [](auto& a, auto& b){ return a.first > b.
        first; });
```

```
10
11      Matrix V(n, n, 0.0);
12      vector<double> sigmas(n);
13      for (int j = 0; j < n; j++) {
14          int idx = sp[j].second;
15          for (int i = 0; i < n; i++) V.A[i][j] = er.vecs.A[i][idx]; //
                reorder V cols
16          sigmas[j] = sqrt(max(0.0, sp[j].first));   // sigma_j = sqrt(
                lambda_j)
17      }
18
19      // Build Sigma (m x n diagonal)
20      Matrix S = zeros(m, n);
21      for (int i = 0; i < rr; i++) S.A[i][i] = sigmas[i];
22
23      // Compute U columns: u_j = A v_j / sigma_j
24      Matrix U(m, m, 0.0);
25      int rank_A = 0;
26      for (int j = 0; j < rr; j++) {
27          if (sigmas[j] > EPS) {
28              for (int i = 0; i < m; i++) {
29                  double s = 0.0;
30                  for (int k = 0; k < n; k++) s += A[i][k] * V.A[k][j];
31                  U.A[i][rank_A] = s / sigmas[j];   // u = Av / sigma
32              }
33              rank_A++;
34          }
35      }
36
37      // Complete U: add orthonormal basis for null(A^T) using Gram-
            Schmidt
38      // against the already-computed u columns
39      vector<vector<double>> u_basis;
40      for (int j = 0; j < rank_A; j++) { /* collect existing U columns */
            }
41      for (int k = 0; k < m && cur_col < m; k++) {
42          vector<double> e(m, 0.0); e[k] = 1.0;  // try each standard
                basis vector
43          // Orthogonalize e against all vectors in u_basis
44          for (auto& u : u_basis) {
45              double dot = ...; for (...) e[i] -= dot * u[i];
46          }
47          double norm = ...; if (norm < EPS) continue;
48          // Accept, normalize, add to u_basis and U
49      }
50      return {U, S, V};
51  }
```

SVD computation chain

1. Form $B = A^T A$ (symmetric, positive semi-definite).
2. Call `_eigenSymmetric()` on B. Returns eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$ and eigenvectors (columns of V).
3. $\sigma_j = \sqrt{\lambda_j}$. Build Σ (diagonal, $m \times n$).
4. For each $\sigma_j > \varepsilon$: $u_j = Av_j/\sigma_j$. This is the relationship $Av = \sigma u$.
5. Complete U to $m \times m$ orthogonal by Gram-Schmidt: try each standard basis vector e_k, orthogonalize against the already-computed u_j's; keep it if the norm after orthogonalization is $> \varepsilon$.

12 Pseudoinverse

```
Matrix Matrix::pinv() const {
    auto [U, S, V] = SVD();
    Matrix Sp = zeros(n, m);       // Sigma^+ is n x m (transposed shape)
    for (int i = 0; i < min(m,n); i++)
        if (S.A[i][i] > EPS) Sp.A[i][i] = 1.0 / S.A[i][i];  // invert
            nonzero sigmas
    return V * Sp * U.T();         // A^+ = V Sigma^+ U^T
}
```

Shape of Σ^+: Σ is $m \times n$; its pseudoinverse Σ^+ is $n \times m$ (transposed shape). If $\Sigma_{ii} = \sigma_i$, then $\Sigma^+_{ii} = 1/\sigma_i$ (for nonzero σ_i). This is then used in $V(n \times n) \cdot \Sigma^+(n \times m) \cdot U^T(m \times m) = A^+(n \times m)$.

13 Fundamental Subspaces

13.1 `nullSpace()` — from free variables in RREF

```
Matrix Matrix::nullSpace() const {
    auto [R, pivot_cols, rank_] = rref();
    int num_free = n - rank_;
    if (num_free == 0) return Matrix(n, 0);   // trivial null space

    // Mark which columns are pivot vs free
    vector<bool> is_pivot(n, false);
    for (int c : pivot_cols) is_pivot[c] = true;
    vector<int> free_cols;
    for (int j = 0; j < n; j++) if (!is_pivot[j]) free_cols.push_back(j)
        ;

    Matrix N(n, num_free, 0.0);
    for (int fc = 0; fc < num_free; fc++) {
        int fc_col = free_cols[fc];
        N.A[fc_col][fc] = 1.0;    // set free variable = 1
        for (int pr = 0; pr < rank_; pr++)
            N.A[pivot_cols[pr]][fc] = -R.A[pr][fc_col];  // pivot = -(
                RREF entry)
    }
    return N;
}
```

From the RREF R, each free column f gives one null-space vector. The equation $Rx = 0$ means: for each pivot row p_k: $x_{p_k} + \sum_{\text{free } f} R_{p_k,f} x_f = 0$. Setting $x_f = 1$ (for this specific free column) and all other free variables to 0 gives $x_{p_k} = -R_{p_k,f}$.

Example for RREF $\begin{pmatrix} 1 & 0 & -1 & -2 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix}$:

- Pivot cols: $\{0, 1\}$. Free cols: $\{2, 3\}$.
- Free col 2 ($x_2 = 1, x_3 = 0$): $x_0 = -(-1) = 1$, $x_1 = -(2) = -2$. Vector: $[1, -2, 1, 0]^T$.
- Free col 3 ($x_2 = 0, x_3 = 1$): $x_0 = -(-2) = 2$, $x_1 = -(3) = -3$. Vector: $[2, -3, 0, 1]^T$.

13.2 `columnSpace()` — original pivot columns

```
Matrix Matrix::columnSpace() const {
    auto [R, pivot_cols, rank_] = rref();
    Matrix C(m, rank_);
    for (int j = 0; j < rank_; j++)
        for (int i = 0; i < m; i++)
            C.A[i][j] = A[i][pivot_cols[j]];    // original A, not R
    return C;
}
```

Why use original A, not RREF R? The pivot columns of R always have 1 in one row and 0 elsewhere (standard unit vectors). That's not a useful basis for the column space of the original A. The pivot columns of the *original* A span the column space and represent geometrically meaningful directions.

13.3 `rowSpace()` — pivot rows of RREF as columns

```
Matrix Matrix::rowSpace() const {
    auto [R, pivot_cols, rank_] = rref();
    Matrix RS(n, rank_);
    for (int i = 0; i < rank_; i++)
        for (int j = 0; j < n; j++)
            RS.A[j][i] = R.A[i][j];    // transpose: row i of R -> column
                i of RS
    return RS;
}
```

The nonzero rows of RREF are the canonical basis for the row space. They are transposed (stored as *columns* of the returned matrix) so the conventions match: all subspace bases are returned as matrices whose *columns* span the subspace.

13.4 `leftNullSpace()` — one-liner

```
Matrix Matrix::leftNullSpace() const { return T().nullSpace(); }
```

$\mathcal{N}(A^T)$ is the null space of A^T. Transpose A, call `nullSpace()`, done.

13.5 Orthonormal versions

```
Matrix Matrix::orthNullSpace() const {
    auto N = nullSpace();
    return (N.n == 0) ? N : N.gramSchmidt();
}
```

The pattern is the same for all four: compute the (possibly non-orthogonal) basis, then call `gramSchmidt()` on it if it is non-trivial. The `N.n == 0` guard handles the trivial (zero) subspace.

14 Projection Matrices

```
Matrix Matrix::projOntoColSpace()  const { return (*this) * pinv(); }
Matrix Matrix::projOntoRowSpace()  const { return T().projOntoColSpace()
    .T(); }
Matrix Matrix::projOntoNullSpace() const { return eye(m) -
    projOntoColSpace(); }
```

Why these formulas work

Onto column space: $P_C = AA^+$. For any b: $P_C b = AA^+ b = A\hat{x}$ where $\hat{x} = A^+ b$ is the least-squares solution. $A\hat{x}$ is the closest point in (A) to b.
Onto null space: $P_N = I - P_C$. Any vector decomposes as $b = P_C b + P_N b$, the column-space component plus the null-space component.
Onto row space: The row space of A is the column space of A^T. So $P_R = (A^T)(A^T)^+ = A^T(A^{T+})$. But $(A^T)^+ = (A^+)^T$ (a property of the pseudoinverse), so $P_R = A^T(A^+)^T = (A^+A)$. In code: `T().projOntoColSpace().T()` computes $(A^T(A^T)^+)^T = (A^+A) = P_R$.

15 Solve `Ax = b`

```
Matrix Matrix::solve(const Matrix& b) const {
    if (b.n != 1 || b.m != m) throw runtime_error("solve: dimension
        mismatch");
    Matrix x = pinv() * b;
    // Snap near-zero solution components to exact zero
    for (int i = 0; i < x.m; i++)
        for (int j = 0; j < x.n; j++)
            if (abs(x.A[i][j]) < EPS) x.A[i][j] = 0.0;
    return x;
}
```

The minimum-norm least-squares solution is $\hat{x} = A^+ b$. This works in all cases:

- A full rank, square: $A^+ = A^{-1}$, so $\hat{x} = A^{-1}b$ (exact solution).
- $b \in (A)$, A rank-deficient: $\hat{x}$ is the minimum-norm exact solution.
- $b \notin (A)$: $\hat{x}$ minimises $\|Ax - b\|_2$.

16 Jordan Form

```cpp
JordanInfo Matrix::jordanForm() const {
    auto er = eigen();
    vector<bool> seen(nn, false);

    for (int i = 0; i < nn; i++) {
        if (seen[i]) continue;
        seen[i] = true;
        int cnt = 1;
        double scale_i = max(1.0, abs(er.real_vals[i]));
        // Cluster nearby eigenvalues into one group
        for (int j = i+1; j < nn; j++) {
            if (!seen[j]
                && abs(er.real_vals[j] - er.real_vals[i]) < 1e-3 *
                    scale_i
                && abs(er.imag_vals[j] - er.imag_vals[i]) < 1e-3 *
                    scale_i) {
                seen[j] = true; cnt++;
            }
        }
        alg_m.push_back(cnt);   // algebraic multiplicity

        // Geometric multiplicity: dim null(A - lambda*I) = n - rank(A -
            lambda*I)
        if (abs(er.imag_vals[i]) < EPS) {
            Matrix AlamI = *this;
            for (int k = 0; k < nn; k++) AlamI.A[k][k] -= er.real_vals[i
                ];
            int gm = max(1, nn - AlamI.rank());   // rank() calls rref()
                internally
            geo_m.push_back(gm);
        } else {
            geo_m.push_back(1);  // complex: assume one block
        }
    }

    // Build Jordan form matrix J block by block
    int pos = 0;
    for (int k = 0; k < uniq_r.size(); k++) {
        int am = alg_m[k], nb = geo_m[k];   // nb = num blocks = geo
            mult
        int base = am / nb, extra = am % nb;
        for (int b = 0; b < nb; b++) {
            int bsz = base + (b < extra ? 1 : 0);  // distribute am into
                nb blocks
            for (int r = 0; r < bsz; r++) {
                J.A[pos+r][pos+r] = lam;               // diagonal =
                    lambda
                if (r < bsz-1) J.A[pos+r][pos+r+1] = 1.0; //
                    superdiagonal = 1
            }
            pos += bsz;
        }
    }
}
```

Given algebraic multiplicity m_a and geometric multiplicity (number of blocks) m_g:

- m_a divided equally into m_g blocks: base size $= \lfloor m_a/m_g \rfloor$, first `extra = m_a % m_g` blocks get $+1$.
- Example: $m_a = 5$, $m_g = 2 \Rightarrow$ blocks of size 3 and 2.
- Example: $m_a = 3$, $m_g = 1 \Rightarrow$ one block of size 3 (single Jordan chain, the defective case).

17 Change of Basis and Similar Matrices

```
Matrix Matrix::changeOfBasis(const Matrix& P) const {
    return P.inv() * (*this) * P;   // P^{-1} A P
}

bool Matrix::isSimilarTo(const Matrix& B) const {
    auto sort_ev = [](vector<double> r, vector<double> im) {
        vector<pair<double,double>> v;
        for (int i = 0; i < r.size(); i++) v.push_back({r[i], im[i]});
        sort(v.begin(), v.end());
        return v;
    };
    auto ea = eigen(), eb = B.eigen();
    if (ea.real_vals.size() != eb.real_vals.size()) return false;
    auto sa = sort_ev(ea.real_vals, ea.imag_vals);
    auto sb = sort_ev(eb.real_vals, eb.imag_vals);
    for (int i = 0; i < sa.size(); i++) {
        if (abs(sa[i].first  - sb[i].first)  > 1e-5) return false;
        if (abs(sa[i].second - sb[i].second) > 1e-5) return false;
    }
    return abs(det() - B.det()) < 1e-5;
}
```

`isSimilarTo` sorts both eigenvalue lists (as complex pairs by real part) and compares them element-wise. The determinant is also compared as an extra check. This is a necessary condition, not sufficient in general, but works for the matrices typical in a linear algebra course.

18 Function Call Graph

The diagram below shows which functions call which others. Arrows mean “calls”.

Caller	Callees
`solve`	`pinv`
`pinv`	`SVD`
`SVD`	`T`, `operator*`, `_eigenSymmetric`, `zeros`, `gramSchmidt*`
`_eigenSymmetric`	`QR`, `operator*`
`_eigenGeneral`	`operator*` (F-L), `nullSpace` (eigenvectors)
`eigen`	`isSymmetric`, `_eigenSymmetric` or `_eigenGeneral`
`jordanForm`	`eigen`, `rank`
`rank`	`rref`
`nullSpace`	`rref`
`columnSpace`	`rref`
`rowSpace`	`rref`
`leftNullSpace`	`T`, `nullSpace`
`orth*Space`	`*Space`, `gramSchmidt`
`projOntoColSpace`	`operator*`, `pinv`
`projOntoRowSpace`	`T`, `projOntoColSpace`
`projOntoNullSpace`	`eye`, `projOntoColSpace`
`det`	(self-contained)
`inv`	`isSingular` (→ `det`)
`QR`	`zeros`, `operator*` (implicitly)
`LUP`	`eye`, `isSquare`
`isPositiveDefinite`	`isSymmetric`, `_eigenSymmetric`
`changeOfBasis`	`inv`, `operator*`
`isSimilarTo`	`eigen`, `det`

`rref()`

Almost every subspace function calls `rref()`. Since `rref()` makes a *copy* (`Matrix R = *this`) it never mutates the original object, so repeated calls are safe (though not cached).

19 main.cpp — Program Flow

19.1 Input phase

```
int m, n;
cout << "Enter matrix dimensions (rows cols): ";
cin >> m >> n;
Matrix A = Matrix::fromInput(m, n);
```

`fromInput` reads each entry from `cin`. After this, `A` is the only data structure; everything else is derived from it.

19.2 Output sections — always computed

Regardless of shape ($m \times n$, any rank):

1. **RREF + pivots:** `auto rres = A.rref();`
2. **Four fundamental subspaces + orthonormal bases.**
3. **QR** (only if $m \geq n$): `auto [Q,R] = A.QR();`

4. **SVD**: `auto [U,S,V] = A.SVD();`
5. **Projection matrices:** `projOntoColSpace`, `projOntoNullSpace`.

19.3 Square-matrix section

Gated on `if (A.isSquare())`:

1. **LUP**: `auto [L,U,P] = A.LUP();`
2. **Properties**: symmetric, orthogonal, singular, PD, PSD.
3. **Determinant**: `A.det();`
4. **Inverse or pseudoinverse**: `A.inv()` if not singular, else `A.pinv()`.
5. **Eigenvalues/vectors**: `auto er = A.eigen();`
6. **Jordan form**: `auto ji = A.jordanForm();`
7. **Diagonalization demo**: if all $m_a = m_g$ and $n \leq 5$, compute $P^{-1}AP$ with $P =$ eigenvector matrix.

For *non-square* matrices, the pseudoinverse and row-space projection are additionally shown.

19.4 Solve phase

```
cout << "Enter column vector b (" << m << " entries): ";
Matrix b_vec(m, 1);
for (int i = 0; i < m; i++) cin >> b_vec.A[i][0];

Matrix x_sol = A.solve(b_vec);
// Compute and print residual
Matrix residual = A * x_sol - b_vec;
double res = 0.0;
for (int i = 0; i < m; i++) res += residual.A[i][0] * residual.A[i][0];
res = sqrt(res);
```

After displaying all structural properties, the program asks for b and calls `solve`. The residual $\|Ax - b\|_2$ confirms whether an exact solution was found or the result is only least-squares.

20 Numerical Constants and Tolerances Summary

Constant	Value	Used for
`EPS`	10^{-9}	Zero threshold everywhere (pivots, norms, det)
`1e-6`	10^{-6}	`approxEqual` (orthogonality check)
`1e-5`	10^{-5}	`isSimilarTo` eigenvalue comparison
`1e-3 * scale_i`	relative	Jordan form eigenvalue clustering
`1e-4 * |re|`	relative	Classifying complex vs real eigenvalue
`1e-12`	10^{-12}	Durand-Kerner convergence (max correction)
`1e-15`	10^{-15}	Durand-Kerner denominator guard
`12000 * n`	—	Max QR iterations for symmetric eigenvalues
`1000 * n`	—	Max Durand-Kerner iterations

21 Known Limitations in the Code

1. **No caching:** `rref()` is called independently by `nullSpace`, `columnSpace`, `rowSpace`, and `rank`. For large matrices this is wasteful; a memoised RREF result would help.
2. **No complex eigenvectors:** `_eigenGeneral` skips eigenvectors for complex eigenvalues (`if (abs(imag) > 1e-7) continue`). A full treatment would use $\mathbb{C}$ arithmetic.
3. **Rank of $(A - \lambda I)$ via RREF:** For nearly-repeated eigenvalues, the floating-point λ may not be accurate enough for the RREF rank to be correct. The `max(1, nn - rank)` guard prevents geo_mult from being zero, but the Jordan block structure may be wrong if eigenvalues are very close.
4. $O(n^4)$ **worst case:** `jordanForm` calls `rank` (which calls `rref`, $O(n^3)$) once per unique eigenvalue. For a matrix with n distinct eigenvalues, total cost is $O(n^4)$.
5. **QR on square submatrix:** `_eigenSymmetric` passes the full $n \times n$ matrix to `QR` at every iteration. A proper implementation would reduce to tridiagonal form once and then do QR steps on a tridiagonal, saving a factor of n.